THESIS DEFENSE PREPARATION · UPDATED EDITION

SLM Code Study Guide

How the SLM follows the adviser's pipeline: every box traced to the file and lines of code that run it, how every file works, a real traced conversation, and questions to prepare for, explained in plain language.

Two-way ASL ↔ English Small Language Model
Updated 24 September 2026, after the code was restructured to follow the adviser's pipeline
Based on the actual code and real test runs on the trained models · Written with Claude (AI assistant)

Files covered (17): `vocab_pool.py`, `tokenizer.py`, `onnx_config.py`, `generate_dataset_v4.py`, `make_labels.py`, `encoder_distilbert.py`, `decoder_tinyllama.py`, `word_picker.py`, `train_distilbert.py`, `train_tinyllama.py`, `train_picker.py`, `slm_pipeline.py`, `speech_input.py`, `full_demo.py`, `two_way_demo.py`, `Conversation_test.py`, `compare.py`

Contents

Before you start

Part 1: Big picture

- What the whole project does
- The trained models in the pipeline
- Which file is the entry point?

Part 2: The pipeline, box by box

- Map: every box and where it lives
- Box: Voice Recording
- Box: Speech to text
- Box: Normalization
- Box: Reduction
- Box: Tokenize
- Box: Embedding
- Box: Contextualize
- Box: Space Restriction
- Box: Active Context (hearing → Deaf)
- Box: Transcribed Words and Display (hearing → Deaf)
- Box: Transcribed Words (Deaf → hearing)
- Box: Active Context (Deaf → hearing)
- Box: Sentence Completion
- Box: Display (Deaf → hearing)

Part 3: How the files connect

- The layers
- Diagram 1: building the models
- Diagram 2: the pipeline (slm_pipeline.py)
- Every file: its role, what it uses, and who uses it
- Why the code is split this way (reasons to give your adviser)

Part 4: File by file

- 4.1 vocab_pool.py: the word lists (lookup table)
- 4.2 tokenizer.py: words ↔ numbers
- 4.3 onnx_config.py: synonyms and input checks
- 4.4 generate_dataset_v4.py: the training-data generator
- 4.5 make_labels.py: keep/drop labels for the word picker
- 4.6 encoder_distilbert.py: the encoder layers
- 4.7 decoder_tinyllama.py: the Sentence Completion model
- 4.8 word_picker.py: the word picker and the Space Restriction rules
- 4.9 train_distilbert.py: teaches Restructure (not in the pipeline now)
- 4.10 train_tinyllama.py: teaches Sentence Completion
- 4.11 train_picker.py: teaches the word picker
- 4.12 slm_pipeline.py: the adviser's pipeline (new)
- 4.13 speech_input.py: Voice Recording and Speech to text (new)
- 4.14 full_demo.py: the live demo with every box traced
- 4.15 two_way_demo.py: the compact demo
- 4.16 Conversation_test.py: scripted conversations against the dataset
- 4.17 compare.py: Reduction accuracy on held-out sentences

Part 5: Full walkthrough of one conversation

- Turn 1: the hearing person says "Where is your mother?"
- Turn 2: the Deaf person signs "home"
- The rest of the conversation

Part 6: Defense preparation

- A 1-minute summary you can say
- 10 questions your adviser might ask
- Weak spots: what was fixed and what is still open
- What to do next

Before you start

This guide explains the SLM code in the main `slm_training_v2` folder. It's based **only** on the actual code: all 17 `.py` files were read line by line. The trained models were also run with read-only checks, so every number and example output here was measured, not guessed.

This is the updated edition (24 September 2026). The code has been restructured to follow your adviser's pipeline diagram exactly. Part 2 shows, box by box, which file and which lines of code run each box.

Project context

- **What it does:** the language part (the SLM) of a two-way ASL ↔ English system. It turns a hearing person's speech into the key signs for the Deaf person, and turns the signs recognised by Veron's camera module into an English sentence for the hearing person.
- **Language and libraries:** Python 3.11 and PyTorch (the neural networks). SciPy is used for exactly one function, `linear_sum_assignment` (the Hungarian algorithm). Vosk is prepared for speech-to-text but not installed yet. Python's built-in `json`, `random` and `re` handle the rest.
- **Reader level:** written for a beginner.

What changed in this update

1. **The code now follows the adviser's pipeline.** A new file, `slm_pipeline.py`, has one method per box, run in the diagram's order. The demos and the conversation test run through it.
2. **Restructure is no longer in the pipeline**, per the adviser. `slm_inference.py`, the old flow that used it, was moved to `archive/`. The Restructure files (`train_distilbert.py`, `trained_distilbert.pt`) are still in the folder in case it comes back. Removing it changed no sentence: TinyLlama wrote the identical sentence for all 979 sign inputs, because it doesn't read sign order.
3. **The two models were split into the diagram's stages, with the same trained weights.** `word_picker.py` now runs as Embedding → Contextualize → Space Restriction. `decoder_tinyllama.py` now runs as Embedding → Sentence Completion. Outputs were verified identical to the old code: 827 of 827 picker sentences and 979 of 979 TinyLlama sentences. Nothing needed retraining.
4. **Six bugs were fixed along the way:**
   - punctuation (`Where is mother?` used to give just `where`)
   - apostrophes (`don't` shifted every word's score)
   - input longer than 16 words crashing
   - the "your" memory bug
   - place answers (`home` after "where is your mother")
   - memory reset
5. **Speech to text:** `speech_input.py` is ready for Vosk, an offline pre-trained model, but it isn't installed yet. That waits for your group's decision. Until then, the hearing person's sentence is typed.

Still true, and worth knowing before your defense (details in Part 6):

1. **TinyLlama can't see the order of the signs**, so "mother help father" and "father help mother" give the same sentence.
2. **The held-out test set is almost all Reduction rows:** 294 Reduction rows, but only 46 Sentence Completion rows.
3. **Some boxes contain hand-written rules, not learning.** Part 2 says which is which for every box.

Part 1: Big picture

What the whole project does

The system has two paths. They match your adviser's diagram exactly.

Hearing → Deaf (single solid line in the diagram). The hearing person speaks, and the Deaf person reads the key signs:

Voice Recording → Speech to text → Normalization → Reduction → Tokenize → Embedding → Contextualize → Space Restriction → Active Context → Transcribed Words → Display

Real example: `Where is your mother?` → **`where mother`**

Deaf → hearing (solid line with broken lines). The Deaf person signs, and the hearing person reads a sentence:

Transcribed Words → Tokenize → Embedding → Active Context → Sentence Completion → Display

Real example: signs `home`, right after the question above → **`mother is home`**

The trained models in the pipeline

Model	Direction	Pipeline boxes it runs	Design file	Trained file	Size
Word picker	hearing → Deaf	Embedding, Contextualize, Space Restriction	`word_picker.py` (body from `encoder_distilbert.py`)	`trained_picker.pt`	75,219 parameters
TinyLlama decoder	Deaf → hearing	Embedding, Sentence Completion	`decoder_tinyllama.py`	`trained_tinyllama.pt`	132,800 parameters
Restructure (DistilBERT)	not in the pipeline now	none	`encoder_distilbert.py`	`trained_distilbert.pt`	75,024 parameters

Both pipeline models were trained from scratch on data your own script generates. (For comparison, the real DistilBERT has about 66 million parameters.)

Which file is the entry point?

Building the models (rerun whenever the vocabulary changes):

```
python generate_dataset_v4.py   # 1. create the training data
python train_tinyllama.py       # 2. train Sentence Completion
python make_labels.py           # 3. turn Reduction rows into keep/drop labels
python train_picker.py          # 4. train the word picker
```

`train_distilbert.py` is only needed if Restructure comes back.

Using the models:

- `python full_demo.py`: a live conversation that prints every pipeline box as it runs. Use this with your adviser.
- `python two_way_demo.py`: the same pipeline, showing only the two screens.

- `python slm_pipeline.py`: runs a short example conversation through both paths.
- `python compare.py` and `python Conversation_test.py`: measure accuracy.

Part 2: The pipeline, box by box

This part answers "which part of the code does each box?" For every box you get: **where** it is (file, function, lines), **the code**, **how it works** in plain words, and **a real example**. Line numbers refer to the current files.

Map: every box and where it lives

Box	Path	Pipeline method (`slm_pipeline.py`)	Where the real work happens
Voice Recording	hearing → Deaf	`voice_recording()`, lines 232–234	`speech_input.py`, `record()`, lines 45–56
Speech to text	hearing → Deaf	`speech_to_text()`, lines 236–238	`speech_input.py`, `transcribe()`, lines 59–97
Normalization	hearing → Deaf	`normalization()`, lines 240–267	same method
Reduction	hearing → Deaf	`reduction()`, lines 269–275	same method
Tokenize	both	`tokenize()`, lines 338–340	`tokenizer.py`, the table built in lines 28–31
Embedding	both	`embedding()`, lines 342–349	speech: `word_picker.py` `embed()` → `encoder_distilbert.py` `DistilBertEmbeddings`, lines 7–42. Signs: `decoder_tinyllama.py` `embed_source()`, lines 239–241
Contextualize	hearing → Deaf	`contextualize()`, lines 277–282	`word_picker.py` `contextualize()`, lines 98–103 → `encoder_distilbert.py` `TransformerBlock`, lines 110–133
Space Restriction	hearing → Deaf	`space_restriction()`, lines 284–293	`word_picker.py` `heads()`, lines 105–115, and `restrict_to_signs()`, lines 134–191
Active Context	hearing → Deaf	`ActiveContext.resolve_pronoun()`, lines 113–127, and `note_hearing()`, lines 129–143	same class
Transcribed Words	hearing → Deaf	`transcribed_words()`, lines 295–297	same method
Transcribed Words	Deaf → hearing	`transcribed_signs()`, lines 301–314	same method
Active Context	Deaf → hearing	`ActiveContext.fill_subject()`, lines 147–169, plus lines 446–449	same class
Sentence Completion	Deaf → hearing	`sentence_completion()`, lines 316–336	`decoder_tinyllama.py` `generate_from_memory()`, lines 271–287, and `decode()`, lines 243–257
Display	both	`display()`, lines 351–354	the screens printed by `full_demo.py` / `two_way_demo.py`

The two paths themselves are the methods `non_deaf_to_deaf()` (lines 358–424) and `deaf_to_non_deaf()` (lines 426–459). They simply call the boxes in the diagram's order:

```
words, changes = self.normalization(text)
kept, removed = self.reduction(words)
ids = self.tokenize(kept)
vectors = self.embedding(ids, "speech")
hidden = self.contextualize(vectors)
glosses, labels, is_question = self.space_restriction(hidden, kept)
glosses, resolved = self.context.resolve_pronoun(words, glosses)
self.context.note_hearing(words, glosses)
out = self.transcribed_words(glosses)
self.display(turn, out)
```

That's the hearing → Deaf path, after Voice Recording and Speech to text.

Box: Voice Recording

Where: `slm_pipeline.py`, `voice_recording()` (lines 232–234), which calls `speech_input.py`, `record()` (lines 45–56).

```
frames = sd.rec(int(seconds * SAMPLE_RATE), samplerate=SAMPLE_RATE,
                channels=1, dtype="int16")
sd.wait()
return frames.tobytes()
```

How it works:

- `sd.rec` records from the microphone for `seconds` (5 by default) at 16,000 samples per second, which is the rate the speech model expects. `channels=1` means mono, and `int16` means each sample is a whole number.
- `sd.wait()` waits until the recording is finished.
- `.tobytes()` hands the raw audio to the next box.

`sd` is the `sounddevice` library.

Status: ready, but waiting for your group's decision. `sounddevice` isn't installed, so `full_demo.py` uses typed text instead, and the trace shows "skipped: typed input".

Box: Speech to text

Where: `slm_pipeline.py`, `speech_to_text()` (lines 236–238), which calls `speech_input.py`, `transcribe()` (lines 59–97).

```
_model = Model(MODEL_DIR)
recognizer = KaldiRecognizer(_model, SAMPLE_RATE)
for start in range(0, len(audio), chunk):
    if recognizer.AcceptWaveform(audio[start:start + chunk]):
        texts.append(json.loads(recognizer.Result()).get("text", ""))
texts.append(json.loads(recognizer.FinalResult()).get("text", ""))
```

How it works:

- `Model(MODEL_DIR)` loads the pre-trained Vosk English model (`vosk-model-small-en-us-0.15`, about 40 MB, offline).
- The audio is fed in chunks. Whenever Vosk detects a pause, it returns a finished phrase.
- The last line collects whatever remains.

- All phrases are joined into the transcript, for example `"where is your mother"`.

There's also an optional `vocabulary` setting that makes Vosk choose only among your lookup-table words. It's off by default, because the adviser's design lets Reduction do that job.

Status: code ready, and untested until Vosk is installed.

Box: Normalization

Where: `slm_pipeline.py`, `normalization()` (lines 240–267).

```
text = normalize_input(text).lower().replace("’", "'")
text = re.sub(r"[^\w\s']", " ", text)
for raw in text.split():
    w = raw.strip("'")
    if w in CONTRACTIONS:
        parts = CONTRACTIONS[w].split()
    elif w.endswith("'s") and known_form(w[:-2]) in LOOKUP_TABLE:
        parts = [known_form(w[:-2])]
    elif w in GLOSS_MAP and w not in picker_training_words():
        parts = [GLOSS_MAP[w]]
    else:
        parts = [w.replace("'", "")]
    words.extend(parts)
```

How it works, line by line:

1. `normalize_input` (from `onnx_config.py`) checks that the input is text, lowercases it, and straightens curly apostrophes.
2. `re.sub(r"[^\w\s']", " ", text)` removes all punctuation except apostrophes. This is what fixed the `"mother?"` bug.
3. Each word then goes through one of four rules:
   - **Short forms are written out** using the `CONTRACTIONS` table: `don't` → `do not`, `i'm` → `i am`, `never` → `not`. The training sentences were written that way.
   - **Possessives lose the `'s`**, and the name gets its trained form: `father's` → `father`, `dad's` → `father`.
   - **A synonym the word picker never saw in training** is replaced by the word it knows, using `GLOSS_MAP`: `mommy` → `mother`, `clothes` → `laundry`. This includes `mom` → `mother`, `dad` → `father` and `house` → `home`. Those three are in the lookup table, but no training sentence ever used them, so the picker never learned them. `picker_training_words()` reads the list of trained words from `labels_train.jsonl`.
   - **Everything else stays as spoken.** Words the picker *was* trained on (like `driving` and `okay`) are not changed, because that's the form it learned them in.

Real examples:

- `"Where did your mommy park the car?"` → `where did your mother park the car` (change: `mommy -> mother`)
- `"I don't know where he is."` → `i do not know where he is` (change: `don't -> do not`)
- `"Is your mom at home?"` → `is your mother at home` (change: `mom -> mother`). The final signs are `mother home`. Before this rule, the untrained word `mom` made the picker answer just `home`.

Box: Reduction

Where: `slm_pipeline.py`, `reduction()` (lines 269–275).

```
kept = [w for w in words if w in LOOKUP_TABLE]
removed = [w for w in words if w not in LOOKUP_TABLE]
return kept, removed
```

How it works: it keeps only the words in the lookup table and removes everything else, **before** tokenizing, as the adviser specified. `LOOKUP_TABLE` is `set(ALL_VOCAB)` from `vocab_pool.py`: 88 words, meaning the 40 signs plus 48 helper words. (The helper list has 54 entries, but six of them are also signs.) The `...` loops are list comprehensions: each builds a new list by testing every word.

Real example: `where did your mother park the car` → kept `where your mother the`, removed `did, park, car`.

Why 88 words and not only the 40 signs? The word picker needs some non-sign words to understand the sentence. For example, `is` at the start is how it knows `is father okay` is a question (→ `how father`). The narrowing down to the 40 signs happens two boxes later, in Space Restriction. Every word in the picker's training sentences is in the lookup table (checked on all 827 labelled rows), so Reduction never removes anything the model learned from.

Box: Tokenize

Where: `slm_pipeline.py`, `tokenize()` (lines 338–340), using the table built in `tokenizer.py` (lines 28–31).

```
return [self.tokenizer.token_to_id[w] for w in words]
```

The table it looks words up in (`tokenizer.py`):

```
all_tokens = SPECIAL_TOKENS + sorted(set(vocab_words))
self.token_to_id = {tok: i for i, tok in enumerate(all_tokens)}
```

How it works: neural networks only work with numbers, so each word becomes its ID. The table lists 4 special tokens first (`<pad>`=0, `<bos>`=1, `<eos>`=2, `<unk>`=3), then the 88 words alphabetically, starting at 4. That's 92 IDs in total. Because Reduction already removed unknown words, every word is guaranteed to have an ID.

Real examples:

- `where is your mother` → `[85, 41, 91, 51]`
- `home` → `[37]`
- `mother laundry` → `[51, 44]`

Box: Embedding

Where: `slm_pipeline.py`, `embedding()` (lines 342–349). It uses a different trained table for each direction:

```
if direction == "speech":
    return self.picker.embed(x)
return self.decoder.embed_source(x)
```

Hearing → Deaf: how the word picker embeds

`word_picker.py` `embed()` (lines 94–96) calls `encoder_distilbert.py` `DistilBertEmbeddings.forward()` (lines 31–42):

```
word_emb = self.word_embeddings(input_ids)
pos_emb = self.position_embeddings(position_ids)
embeddings = word_emb + pos_emb
embeddings = self.layer_norm(embeddings)
embeddings = self.dropout(embeddings)
```

How it works:

1. **Word table.** A learned table of 92 rows × 64 numbers. Each word ID picks its row; that row is the word's "meaning" as the model learned it.
2. **Position table.** A second learned table of 16 rows × 64 numbers, one row per position in the sentence (0–15).
3. **Add them together.** The word's 64 numbers now say both *what* it is and *where* it is.
4. **LayerNorm** rescales the numbers to a steady range.
5. **Dropout** is switched off when the model is being used (`model.eval()`).

Real numbers from your trained model, for `mother` (ID 51) as the 4th word (position 3) of `where is your mother`. Only the first 4 of 64 numbers are shown:

Step	Numbers
word table, row 51 (`mother`)	`[+0.06, -0.73, +1.31, +1.01, ...]`
position table, row 3	`[+1.60, -0.79, -0.71, +0.58, ...]`
added together	`[+1.66, -1.52, +0.61, +1.59, ...]`
after LayerNorm = **Embedding output**	`[+1.40, -1.71, +0.65, +1.48, ...]`

The whole sentence becomes **4 words × 64 numbers**.

Deaf → hearing: how TinyLlama embeds the signs

`decoder_tinyllama.py`, `embed_source()` (lines 239–241):

```
return self.src_embedding(input_ids)
```

How it works: a plain lookup in TinyLlama's own learned table (92 × 64). **No position is added.** Real numbers for the sign `home` (ID 37): `[-1.17, -1.76, -0.05, -0.42, ...]`.

This missing position is why TinyLlama can't see sign order, and why removing Restructure changed no sentence.

Box: Contextualize

Where: `slm_pipeline.py`, `contextualize()` (lines 277–282) → `word_picker.py`, `contextualize()` (lines 98–103):

```
for layer in self.layers:
    x = layer(x, key_padding_mask=padding_mask)
return x
```

Each `layer` is a `TransformerBlock` from `encoder_distilbert.py` (lines 125–133):

```
attn_out = self.attention(self.norm1(x), key_padding_mask)
x = x + self.dropout(attn_out)
ffn_out = self.ffn(self.norm2(x))
x = x + self.dropout(ffn_out)
```

The core of `self.attention` (`MultiHeadSelfAttention`, lines 66–90):

```
scores = torch.matmul(q, k.transpose(-2, -1)) / scale
attn_weights = F.softmax(scores, dim=-1)
context = torch.matmul(attn_weights, v)
```

How it works:

- The word picker has **2 transformer blocks**. In each one, **self-attention** lets every word look at every other word:
  - Each word makes a query ("what am I looking for?"), a key ("what do I contain?") and a value ("what do I pass on?").
  - Queries are compared with keys to score how relevant each word is to each other word.
  - Softmax turns those scores into percentages.
  - Each word becomes a blend of the others' values.
- This runs as 4 "heads" in parallel.
- The **feed-forward** part (64 → 128 → 64) then lets each word think it over alone.
- `x = x + ...` adds a correction instead of replacing the word, which keeps training stable.

Why this box matters: after Contextualize, the same word has **different** numbers depending on the sentence it's in. That's what "context" means here. Real numbers for `mother`:

Sentence	`mother` after Contextualize
`where is your mother`	`[+1.39, -7.89, -1.07, +1.13, ...]`
`mother is sick`	`[-5.23, +3.20, -1.38, -0.27, ...]`

That's why the picker can drop `is` in one sentence but use it to spot a question in another.

Box: Space Restriction

Where: `slm_pipeline.py`, `space_restriction()` (lines 284–293). It has two parts: the model's **scores** (`word_picker.py heads()`, lines 105–115), then the **rules** (`word_picker.py restrict_to_signs()`, lines 134–191).

Part 1, the scores:

```
tag_logits = self.tag_head(x)
pooled = (x * keep).sum(dim=1) / keep.sum(dim=1).clamp(min=1.0)
question_logits = self.question_head(pooled)
```

- `tag_head` gives every word **17 scores**. Score 0 means DROP; scores 1–16 mean "keep it, in slot N".
- `question_head` looks at the **average** of all the words and gives 2 scores: `[normal, add "how"]`. This is for wellbeing questions like `is father okay` → `how father`, where "how" was never said.

Part 2, the rules that narrow everything down to the 40-sign space:

```
if words[i].lower() in NEGATIONS or w in NEGATIONS:
    negated = True
    continue
if w not in SIGNS:
    continue
if scores[i, 1:].max() > scores[i, 0]:
    keep_idx.append(i)
...
rows, cols = linear_sum_assignment(-sub)
```

- **Negation words** (`not`, `don't`, `never`...) are never shown directly. They make the sign `no` appear at the front instead.
- **A word that isn't one of the 40 signs is always dropped**, whatever the model scored. The Deaf person can't be shown a word that has no sign.
- **A sign word is kept** if its best "keep" score beats its DROP score.
- **The Hungarian algorithm** (`linear_sum_assignment`) gives each kept word its own slot, choosing the one-to-one match with the highest total score, so no two words can land in the same slot.

Real scores from your model for `where is your mother`:

Word	DROP score	Best keep score	Decision
where	0.73	slot 1: **22.41**	keep in slot 1
is	**17.81**	lower	drop (also not a sign)
your (sign `you`)	**21.57**	lower	drop
mother	-5.11	slot 2: **14.22**	keep in slot 2

The question scores were `[11.91, -9.16]`, so "normal" wins and no "how" is added. The result is **`where mother`**.

Box: Active Context (hearing → Deaf)

Where: `slm_pipeline.py`, class `ActiveContext`: `resolve_pronoun()` (lines 113–127) and `note_hearing()` (lines 129–143).

```
said = set(spoken_words)
if not any(p in said and self.person in fits
           for p, fits in PRONOUN_FITS.items()):
    return glosses, None
i = 0
while i < len(glosses) and (glosses[i] in QUESTIONS or glosses[i] == "no"):
    i += 1
return glosses[:i] + [self.person] + glosses[i:], self.person
```

How it works:

- **Replacing he/she.** If the hearing person said `he` or `she`, and the memory holds a person that pronoun can refer to, that person is added to the signs. The person goes after a leading question word or `no`, matching the training outputs. `PRONOUN_FITS` says `he` can be father, teacher or friend, and `she` can be mother, teacher or friend. Family is plural, so neither pronoun refers to it.
- **Remembering** (`note_hearing`):
  - It stores the last person mentioned.

- It stores whether the turn was a question, and which question word was used.
- It stores whether the turn was asked **to the Deaf person**. Only the word `you` itself counts; `your father` is about father. This fixed the "your" bug.

Real example: after "Where is your mother?", the hearing person says `Is she okay?`. Space Restriction gives `how`, and Active Context turns it into `how mother`.

Box: Transcribed Words and Display (hearing → Deaf)

Where: `transcribed_words()` (lines 295–297) joins the signs into the text for the Deaf person, for example `where mother`. `display()` (lines 351–354) stores it as the screen output. `full_demo.py` and `two_way_demo.py` draw the screen box.

Box: Transcribed Words (Deaf → hearing)

Where: `slm_pipeline.py`, `transcribed_signs()` (lines 301–314).

```
for w in self.tokenizer._clean_and_split(text):
    s = normalize_word(w)
    if s in SIGNS:
        signs.append(s)
    else:
        ignored.append(w)
```

How it works: it takes the words Veron's sign recognizer produced. Recognizer labels are mapped to sign names with `GLOSS_MAP` (`mama` → `mother`), and anything that isn't one of the 40 signs is set aside.

Real example: `mama laundry xyz` → signs `mother laundry`, with `xyz` set aside.

Box: Active Context (Deaf → hearing)

Where: `slm_pipeline.py`, `ActiveContext.fill_subject()` (lines 147–169). The person's vector is added in `deaf_to_non_deaf()`, lines 446–449.

```
needs_subject = (
    any(s in QUESTIONS for s in signs)
    or (any(s in STATES for s in signs)
        and not any(s in ACTIVITIES for s in signs))
    or ("where" in self.last_question_words
        and all(s in PLACES for s in signs))
)
```

```
extra = self.embedding(self.tokenize([added]), "signs")
vectors = torch.cat([extra, vectors], dim=1)
```

How it works:

- A remembered person is added only when the signs **need a subject** and none was signed. The signs need a subject when:
  - they contain a question word (`where`), or
  - they contain a state (`fine`) without an activity, or

    - they're a bare place (`home`) answering a *where* question.
- It's **not** added if:
    - a person was signed, or
    - `my` was signed (the signer means themself), or
    - the last question was asked to the Deaf person (`how are you`).
- Because this box comes **after** Embedding in the diagram, the person is added as a vector. The person's embedding is joined to the front of the sign vectors (`torch.cat`).

Real example: signs `home` after "Where is your mother?" → `added 'mother' from earlier`.

Box: Sentence Completion

Where: `slm_pipeline.py`, `sentence_completion()` (lines 316–336). It calls `decoder_tinyllama.py`, `generate_from_memory()` (lines 271–287), which uses `decode()` (lines 243–257).

```
generated = torch.tensor([[tokenizer.bos_id]], device=device)
for _ in range(max_new_tokens):
    logits = self.decode(memory, generated, src_padding_mask)
    next_token = logits[0, -1, :].argmax().item()
    if next_token == tokenizer.eos_id:
        break
    generated = torch.cat([generated, torch.tensor([[next_token]], device=device)], dim=1)
```

How it works:

- TinyLlama writes **one word at a time**. It starts with `<bos>`, scores all 92 words for the next position, takes the highest (`argmax`), adds it, and repeats until it writes `<eos>` (at most 16 words).
- Inside `decode()`, each of its 2 blocks does three things:
    - **causal self-attention** with RoPE: a word can only see the words already written
    - **cross-attention**: it reads the sign vectors from Embedding and Active Context
    - a **SwiGLU** feed-forward layer
- Two rules then follow (`sentence_completion`):
    - **Answer form** (`to_declarative`, lines 187–197): a reply to a question is written as a statement, so `is mother home` → `mother is home`.
    - **Safety net** (`covered`, lines 200–207): if the sentence left out a sign, the plain signs are shown instead of a fluent sentence about something else.

Real example: from the vectors for `[mother, home]`, TinyLlama writes `is` (100%) → `mother` (100%) → `home` (100%) → `<eos>`. That gives `is mother home`, and the answer form turns it into **`mother is home`**.

Box: Display (Deaf → hearing)

Where: `display()` (lines 351–354) stores the sentence as the screen output, and the demos draw the "HEARING PERSON'S SCREEN" box.

Part 3: How the files connect

The layers

Each layer only uses the layers below it:

```
LAYER 5  Demos & tests   full_demo.py   two_way_demo.py   Conversation_test.py   compare.py
LAYER 4  Pipeline        slm_pipeline.py   speech_input.py
LAYER 3  Training        train_tinyllama.py   train_picker.py   (train_distilbert.py: Restructure, unused)
LAYER 2  Model designs   decoder_tinyllama.py   word_picker.py   encoder_distilbert.py (picker's body)
LAYER 1  Data            generate_dataset_v4.py   make_labels.py
LAYER 0  Foundation      vocab_pool.py   tokenizer.py   onnx_config.py
```

Diagram 1: building the models

```
generate_dataset_v4.py
        | writes
        ▼
dataset_v4_train.jsonl  (+ dataset_v4_test.jsonl)
        |
        ├── complete rows ──────► train_tinyllama.py ──► trained_tinyllama.pt
        |                          (uses decoder_tinyllama.py)
        |
        ├── reduce rows ────────► make_labels.py ──► labels_train.jsonl (+ labels_test.jsonl)
        |                                                    |
        |                                                    ▼
        |                                          train_picker.py ──► trained_picker.pt
        |                                          (uses word_picker.py)
        |
        └── restructure rows ──► train_distilbert.py ──► trained_distilbert.pt  (not in the pipeline now)
```

Diagram 2: the pipeline (slm_pipeline.py)

```
HEARING -> DEAF                          DEAF -> HEARING
Voice Recording     speech_input.record  Transcribed Words   transcribed_signs()
Speech to text      speech_input.transcribe        |
Normalization       normalization()                |
Reduction           reduction()                    |
Tokenize            tokenize()  ◄── shared ──►  Tokenize            tokenize()
Embedding           picker.embed()                Embedding           decoder.embed_source()
Contextualize       picker.contextualize()                |
Space Restriction   picker.heads() + restrict_to_signs()  |
Active Context      resolve_pronoun(), note_hearing()     Active Context   fill_subject()
Transcribed Words   transcribed_words()                   Sentence Completion  decoder.generate_from_memory()
Display             display()                     Display             display()
```

Every file: its role, what it uses, and who uses it

File	One-line role	Uses (from your project)	Used by
`vocab_pool.py`	**Stores** the word lists (lookup table)	nothing	almost every file
`tokenizer.py`	**Converts** words ↔ numbers	vocab_pool (self-test only)	slm_pipeline, training scripts, compare
`onnx_config.py`	**Rules** for synonyms and input checks (no PyTorch)	vocab_pool	make_labels, slm_pipeline
`generate_dataset_v4.py`	**Creates** the training data	nothing	you run it
`make_labels.py`	**Converts** Reduction rows into keep/drop labels	onnx_config	word_picker, slm_pipeline, compare
`encoder_distilbert.py`	**Defines** the encoder layers (picker's body; also Restructure)	(PyTorch only)	word_picker, train_distilbert
`decoder_tinyllama.py`	**Defines** the Sentence Completion network	(PyTorch only)	slm_pipeline, train_tinyllama, compare
`word_picker.py`	**Defines** the word picker and the Space Restriction rules	make_labels, vocab_pool, encoder_distilbert	slm_pipeline, train_picker, compare
`train_tinyllama.py`	**Trains** Sentence Completion	tokenizer, vocab_pool, decoder_tinyllama	run by you
`train_picker.py`	**Trains** the word picker	tokenizer, vocab_pool, word_picker	run by you
`train_distilbert.py`	**Trains** Restructure (not in the pipeline now)	tokenizer, vocab_pool, encoder_distilbert	run by you, if needed
`slm_pipeline.py`	**Runs** both paths, one method per diagram box	tokenizer, vocab_pool, onnx_config, make_labels, word_picker, decoder_tinyllama, speech_input	full_demo, two_way_demo, Conversation_test
`speech_input.py`	**Records and transcribes** speech (Vosk)	nothing (Vosk, sounddevice when installed)	slm_pipeline, full_demo
`full_demo.py`	**Displays** a live conversation with every box traced	slm_pipeline, speech_input	run by you
`two_way_demo.py`	Same conversation, screens only	slm_pipeline	run by you
`Conversation_test.py`	**Tests** scripted conversations against the dataset	slm_pipeline	run by you
`compare.py`	**Measures** Reduction accuracy on held-out sentences	tokenizer, vocab_pool, word_picker, decoder_tinyllama, make_labels	run by you

Why the code is split this way (reasons to give your adviser)

1. **The code mirrors the pipeline diagram.** One method per box in `slm_pipeline.py`, called in the diagram's order. Anyone can check the code against the diagram line by line.

2. **One source of truth.** The words live only in `vocab_pool.py`, and the synonyms only in `onnx_config.GLOSS_MAP`.
3. **Model design, training and use are separate.** The model files only describe the networks. The training scripts only train them. `slm_pipeline.py` only uses them.
4. **Model stages are visible.** Embedding, Contextualize and Space Restriction are separate methods of the same trained model, so each box can be shown on its own. `forward()` still runs them in order, so training uses exactly the same computation.
5. **The demos contain no logic.** `full_demo.py` and `two_way_demo.py` only display what the pipeline produces, so they can't drift apart.
6. **Reproducible.** All training scripts fix `SEED = 42`.
7. **Ready for the Pi.** `onnx_config.py` has no PyTorch in it, and speech recognition runs offline.

Part 4: File by file

This part covers all 17 files in the main folder, in five groups: **foundation** (4.1–4.3), **data** (4.4–4.5), **models** (4.6–4.8), **training** (4.9–4.11) and **pipeline, demos and tests** (4.12–4.17). Each file gets its purpose, contents, how it works, inputs, outputs, connections, key concepts, and a heads-up about anything that could trip you up. For the pipeline boxes, Part 2 has the code line by line; this part gives each file's overall picture.

4.1 vocab_pool.py: the word lists (lookup table)

1. Purpose. The master word list. Other files ask it "which words exist?" and "which words are signs?"

2. What it contains

Name	Type	What it is
`CONFIRMED_GESTURES`	list of 40 words	The signs the camera can recognise. The **only** words Veron's module can send.
`HELPER_WORDS`	list of 54 words	Words that are never signed
`ALL_VOCAB`	sorted list of 88 words	Both lists merged with repeats removed: **the lookup table**
`CATEGORIES`	dictionary, 40 entries	Each sign mapped to its group, e.g. `"mother": "Person / Family"`

3. How it works. The key line:

```
ALL_VOCAB = sorted(set(CONFIRMED_GESTURES + HELPER_WORDS))
```

- `+` joins the lists: 40 + 54 = 94 words.
- `set(...)` removes repeats. Six words are in both lists (`you, my, his, her, please, want`), so 88 remain.
- `sorted(...)` keeps them in alphabetical order, so every word's ID stays the same on every run.

Why helper words exist. They do two jobs:

- (a) grammar words Sentence Completion writes: `is`, `am`, `are`, `doing`…
- (b) words hearing people say, which the word picker must recognise so it can learn to **drop** them: `the`, `very`, `okay`.

A word that isn't in the vocabulary becomes a generic "unknown" token, and the model can't learn a rule about a word it can't tell apart from every other unknown word.

`CATEGORIES` in action. `slm_pipeline.py` builds its groups from it:

```
PEOPLE = {w for w, c in CATEGORIES.items() if c == "Person / Family"}
```

This gives `{mother, father, teacher, family, friend}`, the people Active Context can remember. Pronouns (`his, her, my, you`) deliberately have their own category, so the memory never treats "you" as the person being talked about. The same pattern builds `QUESTIONS`, `STATES`, `ACTIVITIES` and `PLACES`.

4. Inputs. None; the words are typed by hand. **5. Outputs.** The four variables.

6. Connections. It imports nothing. Almost every file uses it; it's not used by `generate_dataset_v4.py` (see the heads-up).

7. Key concepts

- **List vs. set.** A list keeps order and allows repeats; a set has neither.
- `if __name__ == "__main__":` runs only when you type `python vocab_pool.py` (it prints 40 / 54 / 88).
- **Closed vocabulary.** The system knows exactly these 88 words.

Heads-up

- Comments say "39 signs"; the list has 40.
- A comment says `conversation_session.py` uses `CATEGORIES`. In the main folder, `slm_pipeline.py` uses it.
- **If you add a sign, also add it to `generate_dataset_v4.py`'s category lists**, then regenerate the data and retrain.

4.2 tokenizer.py: words ↔ numbers

1. Purpose. Turns each word into an ID number and back. It provides the Tokenize box's lookup table.

2. What it contains

Name	What it does
`SPECIAL_TOKENS`	`<pad>`, `<bos>`, `<eos>`, `<unk>`
`SimpleTokenizer.__init__(vocab_words)`	Builds the word→ID (`token_to_id`) and ID→word tables
`_clean_and_split(text)`	Lowercases, removes punctuation, splits into words
`encode(text, add_special_tokens=True)`	Text → list of IDs
`decode(ids, skip_special_tokens=True)`	List of IDs → text
`save(path)` / `load(path)`	Writes or reads the table as JSON

3. How it works. The table is built by listing the special tokens first, then the 88 words alphabetically (see Part 2, Tokenize), giving 92 IDs. Some real IDs:

drive	father	fine	home	how	is	mother	where	your
17	23	24	37	39	41	51	85	91

Cleaning:

```
text = text.lower()
text = re.sub(r"[^\w\s]", " ", text)
text = re.sub(r"\s+", " ", text).strip()
```

This lowercases the text, turns punctuation into spaces, and squashes repeated spaces. `encode` looks each word up with `.get(w, self.unk_id)`, so an unknown word becomes `<unk>` (3) instead of crashing. It can also wrap the result in `<bos>` … `<eos>`: `encode("Where is mother?")` → `[1, 85, 41, 51, 2]`.

4. Inputs. A word list (always `ALL_VOCAB`), then text. **5. Outputs.** IDs, text, and `vocab_size` / `pad_id` / `bos_id` / `eos_id`, which the models use to size themselves.

6. Connections. `slm_pipeline` (Tokenize, and cleaning in Transcribed Words), the training scripts and `compare` use it.

7. Key concepts

- **Word-level tokenizer.** One ID per whole word; enough for a small closed vocabulary.
- **Special tokens.** `<pad>` fills short sentences in a training batch. `<bos>` and `<eos>` tell the decoder where to start and stop.

- **Why a vocabulary change means retraining.** Adding one word shifts the IDs of every word after it alphabetically, and changes the size of the model's embedding table.

Heads-up. On its own, the cleaning turns `don't` into `don t`: two unknown words. The pipeline's Normalization box handles this *before* tokenizing, by writing `don't` out as `do not`.

4.3 onnx_config.py: synonyms and input checks

1. Purpose. Despite the name, the parts used today have nothing to do with ONNX. It's the **shared rulebook for synonyms and input checks**, written with zero PyTorch so the Raspberry Pi side can use it too.

2. What it contains

Name	What it does	Used in the main folder?
`GLOSS_MAP`	Synonym and word-form table (`mom` → `mother`, `driving` → `drive`)	Yes: `make_labels`, `slm_pipeline`
`normalize_input`	Checks the input is text (or a list of words) and trims it	Yes: `slm_pipeline`
`KNOWN_WORDS`	`set(ALL_VOCAB)`	Only inside the functions below
`ENCODER_ONNX_PATH`, `DECODER_ONNX_PATH`, `DECODER_TGT_LEN`, `pad_ids`	ONNX export settings	No (Pi files)
`ENCODER_MAX_LEN`, `DECODER_SRC_LEN`, `sanitize_glosses`, `enforce_max_len`	Checks used by the old `slm_inference.py`	No (now in `archive/`)
`map_glosses`, `oov_density`, `reduce_confidence`	Alias mapping, unknown-word ratio	No (Pi files)

3. How it works. `GLOSS_MAP` has two kinds of entry: true synonyms (`"mom": "mother"`, `"house": "home"`, `"okay": "fine"`) and word forms folded onto the sign's name (`"driving": "drive"`, `"your": "you"`, `"hi": "hello"`). Files use it as `GLOSS_MAP.get(w, w)`: "the standard form if there is one, otherwise the word itself". `"good"` is deliberately not a key, because it's its own sign.

`normalize_input`:

```
if text is None:
    raise TypeError(...)
if isinstance(text, (list, tuple)):
    text = " ".join(str(w) for w in text)
if not isinstance(text, str):
    raise TypeError(...)
return text.strip()
```

It gives a clear error for `None` or a number, and joins a list like `["mother", "drive"]` into `"mother drive"`, so Veron's module can send either form.

4–6. It takes raw text or word lists and returns cleaned text and the table. It imports `vocab_pool`.

7. Key concepts. Dictionary lookup with a default (`d.get(key, default)`); defensive programming (fail early with a clear message).

Heads-up. Its docstring still mentions `slm_inference.py`, which is now archived. Several functions are only used by the files in `pi_deployment_later/` and `archive/`.

4.4 generate_dataset_v4.py: the training-data generator

1. Purpose. Builds all training and test examples **from rules over sign categories**, not hand-written sentences. That's your answer to "no fixed sentences": add a sign to a category and every rule covers it.

2. What it contains

Name	What it is
`SEED = 42`	Makes the train/test split identical every run
`QUESTION_MARKER_WORD = "how"`	The one word Reduction may output even though it wasn't spoken
`PERSON, STATE, ACTIVITY, PLACE, THING, QUESTION, SOCIAL, NEED, PRONOUN, POSSESSIVE`	The signs grouped by grammatical role
`VERB`, `PLACE_PHRASE`, `PERSON_ARTICLE`, `SINGLE`	How signs read inside sentences
`PRONOUN_OF`	Defined, never used
`build_complete()`	Rules: signs → sentence (trains Sentence Completion)
`build_reduce()`	Rules: spoken sentence → signs (trains the word picker)
`build_restructure()`	Scrambled → ordered rows (only for Restructure, not in the pipeline)
`split_by_phrasing()`	The train/test split
`check_task_shapes()`, `check_no_contradictions()`	Safety checks that stop the script on bad data

3. How it works. A **pattern** is (a list of inputs, one output). A real rule:

```
for pr in PERSON:
    who = PERSON_ARTICLE[pr]
    for st in STATE:
        add([pr, st], f"{who} is {st}")
```

Five people × six states = 30 patterns. The f-string fills in the words ("mother is sad"), and `add` includes both sign orders (`mother sad`, `sad mother`).

`build_reduce` gives **several phrasings of one meaning**:

```
pats.append(([f"{p} is {s}", f"my {p} is {s}",
              f"your {p} is {s}", f"{p} is very {s}"],
             f"{p} {s}"))
```

All four give `mother sad`, which forces the picker to learn the rule (drop `my / your / is / very`) instead of memorising sentences.

`split_by_phrasing` groups rows by pattern and never holds out a single-sign input (the "water" story in its docstring). Otherwise it holds out up to 2 phrasings per pattern, keeping at least 2 in training.

Your real file sizes:

File	complete	reduce	restructure	total
`dataset_v4_train.jsonl`	935	533	941	**2,409**

File	complete	reduce	restructure	total
`dataset_v4_test.jsonl`	46	294	2	**342**

4–6. It takes no input, writes the two dataset files, and imports only `json` and `random`.

7. Key concepts. Nested loops; f-strings; JSONL; data augmentation; train/test split; seeded randomness.

Heads-up

- The test set is almost all Reduction rows (294). Held-out evidence for Sentence Completion is thin (46 rows).
- It keeps its own copy of the sign lists instead of importing `vocab_pool`.
- One label drops a negation: `i do not understand` → `understand`. The picker keeps negations (`no understand`), so `compare.py` counts it as a miss.
- Restructure rows are still generated, but only `train_distilbert.py` uses them.
- Small leftovers: "39 signs" in the docstrings (it's 40), an empty `# name + PERSON` heading, the unused `PRONOUN_OF`.

4.5 make_labels.py: keep/drop labels for the word picker

1. Purpose. Converts each Reduction row (sentence → signs) into **one label per word**: `0` = drop, `1, 2, 3…` = keep in that slot. There's also one **question flag** per sentence, meaning "put `how` in front".

2. What it contains

Name	What it does
`NORMALIZE`	`onnx_config.GLOSS_MAP` under another name
`QUESTION_MARKER = "how"`	The one word the flag can add
`normalize_word(w)`	`GLOSS_MAP.get(w, w)`, used across the project
`derive_labels(...)`	The core: one label per input word
`label_row(...)`	Tries plain labelling, then the question-flag version
`rebuild(...)`, `verify(...)`	Rebuild each answer from its labels and compare
`main()`	Reads `dataset_v4_*`, writes `labels_*`, prints a report

3. How it works. For `where is your mother` → `where mother`:

```
for out_pos, ow in enumerate(output_words, start=1):
    candidates = [i for i, iw in enumerate(in_words)
                  if not used_input[i] and (iw == ow or normalize_word(iw) == ow)]
    ...
    found = candidates[-1]
    labels[found] = out_pos
```

The labels start at `[0, 0, 0, 0]`. `where` is output word 1, so `labels[0] = 1`. `mother` is output word 2, so `labels[3] = 2`. The result is **`[1, 0, 0, 2]`**, which is exactly what your trained picker predicts for this sentence (Part 2, Space Restriction).

For `is father okay` → `how father`, `how` isn't in the input, so the rest is labelled (`[0, 1, 0]`) and the question flag is set to 1. `verify` rebuilds every answer from its labels, so a labelling mistake is caught before any training.

Real results: 533 training rows (10 with the flag) and 294 test rows (13 with the flag), with none skipped.

4–6. It reads `dataset_v4_*.jsonl`, writes `labels_*.jsonl`, and imports `onnx_config`. `word_picker`, `slm_pipeline` and `compare` import its `normalize_word`.

7. Key concepts. Token classification (labelling each word); round-trip check.

Heads-up. The docstring says it reads `dataset_v3_*`; the code (line 182) reads `dataset_v4_*`.

4.6 encoder_distilbert.py: the encoder layers

1. Purpose. Defines the DistilBERT-style encoder layers. Today they matter mainly as **the word picker's body**: its Embedding and Contextualize boxes use these classes. The file also defines the full Restructure model (`DistilBertForRestructure`), which is not in the pipeline now.

2. What it contains

Name	What it does	Pipeline box
`DistilBertEmbeddings`	Word table + position table, added, then normalised	Embedding (hearing → Deaf)
`MultiHeadSelfAttention`	Every word looks at every other word, 4 heads	Contextualize
`FFN`	64 → 128 → 64 per word, with GELU	Contextualize
`TransformerBlock`	Attention + FFN with residuals, Pre-LN	Contextualize
`DistilBertForRestructure`, `DISTILBERT_CONFIG`	The Restructure model	not in the pipeline

3. How it works. Part 2 walks through the code of `DistilBertEmbeddings.forward()` (lines 31–42), `MultiHeadSelfAttention.forward()` (lines 66–90) and `TransformerBlock.forward()` (lines 125–133), with real numbers. Two extra details:

- **Padding mask:** `scores.masked_fill(key_padding_mask..., float("-inf"))` gives padding 0% attention.
- **Bidirectional:** there's no "future" mask, so every word sees words before *and* after it. That's why an encoder suits keep/drop decisions.

Where the encoder's parameters come from (the picker uses the same body):

Part	Calculation	Parameters
Word table	92 × 64	5,888
Position table	16 × 64	1,024
Embedding LayerNorm	64 + 64	128
2 transformer blocks	2 × (16,640 attention + 16,576 FFN + 256 norms)	66,944
Body total		**73,984**

4–6. It takes token IDs and returns 64 numbers per word. It uses only PyTorch, and is imported by `word_picker.py` and `train_distilbert.py`.

7. Key concepts. `nn.Module` and `forward()`; embedding; attention (Q/K/V); softmax; residual connection; LayerNorm; dropout (off during use, via `eval()`).

Heads-up. The `FFN` docstring says "4x ... we maintain this ratio", but the config uses 2×. The `TransformerBlock` docstring says Pre-LN matches HuggingFace's DistilBERT, but HuggingFace uses Post-LN. Say "we chose Pre-LN for training stability".

4.7 decoder_tinyllama.py: the Sentence Completion model

1. Purpose. Defines the network that **writes a sentence one word at a time** from the signs.

2. What it contains

Name	What it does
`RMSNorm`	Simpler normalisation (Llama's choice)
`precompute_rope_freqs`, `apply_rope`	RoPE: position by rotating pairs of numbers
`SwiGLU`	Gated feed-forward layer
`TinyLlamaCausalAttention`	Each written word sees only earlier words
`CrossAttention`	The sentence being written reads the signs
`TinyLlamaBlock`	norm → causal attention → norm → cross-attention → norm → SwiGLU
`embed_source()` (**new**, lines 239–241)	Embedding box: sign IDs → vectors, no position
`decode()` (**new**, lines 243–257)	Scores the next word, reading the sign vectors
`generate_from_memory()` (**new**, lines 271–287)	Sentence Completion: writes word by word from vectors
`forward()`, `generate()`	Same computation as before, now built from the parts above

3. How it works

- **Split into the diagram's stages.** `forward()` = `decode(embed_source(ids), ...)`, and `generate()` = `generate_from_memory(embed_source(ids), ...)`. The split lets Active Context add a person's vector *between* Embedding and Sentence Completion, as the diagram draws it. Verified: identical outputs to the old code on all 979 sign inputs.
- **RMSNorm** divides the numbers by their root-mean-square, then scales them by a learned weight.
- **RoPE** rotates each pair of query/key numbers by an angle that grows with position, so attention depends on how far apart two words are:

```
x_r[..., 0] * cos - x_r[..., 1] * sin,
x_r[..., 0] * sin + x_r[..., 1] * cos,
```

- **The causal mask** `torch.triu(torch.ones(max_len, max_len), diagonal=1)` marks the "future" so it gets 0% attention.
- **Generation** is greedy: always take the top word (see Part 2).

Where the 132,800 parameters come from:

Part	Calculation	Parameters
Target word table + sign table	2 × 92 × 64	11,776
2 decoder blocks	2 × (16,384 self-attention + 16,384 cross-attention + 24,576 SwiGLU + 192 norms)	115,072
Final RMSNorm + output head	64 + 64 × 92	5,952

Part	Calculation	Parameters
Total		**132,800**

Why removing Restructure changed no sentence

The signs are embedded with **no position** (`embed_source` is a plain lookup), and cross-attention is a weighted **sum** over the sign vectors. Sums don't care about order (a + b = b + a), so the order of the signs is invisible to TinyLlama. Tested: `mother help father` and `father help mother` give **identical** scores, and all 979 sign inputs gave the same sentence with and without Restructure.

The cost: "who did what to whom" is lost. The fix, if it's ever needed, is to add a position table to the signs (as the encoder does) and retrain.

4–6. It takes sign IDs (and, during training, the correct sentence) and returns word scores or a written sentence. It uses only PyTorch, and is imported by `slm_pipeline`, `train_tinyllama` and `compare`.

7. Key concepts. Decoder (autoregressive); causal mask; cross-attention; greedy decoding; no KV-cache (the whole sentence is recomputed at each step, which is fine for short sentences).

4.8 word_picker.py: the word picker and the Space Restriction rules

1. Purpose. Defines the hearing → Deaf model, which scores **each spoken word**: drop it, or keep it in slot N. It also holds the Space Restriction rules that turn those scores into signs.

2. What it contains

Name	What it does
`SIGNS`, `NEGATIONS`	The 40 signs; 14 negating words
`WordPicker`	DistilBERT body + two heads
`embed()` (**new**, lines 94–96)	Embedding box
`contextualize()` (**new**, lines 98–103)	Contextualize box
`heads()` (**new**, lines 105–115)	Space Restriction scores: tag head (17 per word) + question head
`forward()`	`heads(contextualize(embed(ids)))`: the same computation as before
`restrict_to_signs()` (**new**, lines 134–191)	Space Restriction rules: signs only, negation → "no", Hungarian, "how"
`picker_predict()`	The whole Reduction step in one call, used by `compare.py`

3. How it works. Part 2 shows `heads()` and `restrict_to_signs()` line by line, with your model's real scores. The picker has 75,219 parameters: the 73,984-parameter body (4.6), plus the tag head (64 × 17 + 17 = 1,105) and the question head (64 × 2 + 2 = 130).

Bugs from the previous edition, now fixed:

Bug	Before	Now
Punctuation	`Where is mother? Is she home?` → `where`	`where mother home`. Normalization strips punctuation first, and `picker_predict` now cleans words too
Apostrophes	`i don't know where mother is` → `no mother` (scores shifted)	`don't` → `do not` in Normalization; words and scores always line up
17+ words	crashed	cut at 16 words

4–6. It takes a sentence (or vectors, in the pipeline) and returns signs, per-word labels and the question flag. It imports `make_labels`, `vocab_pool`, `encoder_distilbert` and SciPy.

7. Key concepts. Token classification (it can only output words that were actually said, plus "how"); multi-task heads; mean pooling; the Hungarian algorithm; hybrid design (learned scores + fixed rules).

4.9 train_distilbert.py: teaches Restructure (not in the pipeline now)

1. Purpose. Trains the Restructure model and saves `trained_distilbert.pt`. **The pipeline doesn't use it now**, per the adviser. It's kept in case Restructure comes back.

2–3. How it works. It reads the 941 Restructure rows. `build_position_labels` works out each input word's correct position (`sad mother` → `mother sad` gives labels `[1, 0]`), and the model is trained for 100 epochs with the standard 5-step loop (see 4.10).

Heads-up. 718 of its 941 rows are already in order, and no 3-sign row is ever scrambled. That's one more reason Restructure added nothing to the final sentence.

4.10 train_tinyllama.py: teaches Sentence Completion

1. Purpose. Trains TinyLlama to turn signs into sentences, and saves `trained_tinyllama.pt`.

2. What it contains. `set_seed()` (seed 42); settings (`EPOCHS = 150`, `BATCH_SIZE = 8`, `LEARNING_RATE = 1e-3`, `MAX_LEN = 32`); `load_generation_examples(path, tasks=("complete",))`, which keeps **Complete rows only**; `GenerationDataset`; `collate_fn()`; `main()`.

3. How it works. Teacher forcing:

```
tgt_input  = tgt_ids[:, :-1]
tgt_labels = tgt_ids[:, 1:]
```

For the row `mother home` → `is mother home`, the target IDs are `[1, 41, 51, 37, 2]`. The model is given `<bos> is mother home` and must predict `is mother home <eos>`, one position at a time. The training loop:

```
optimizer.zero_grad()     # 1. clear old corrections
logits = model(src_ids, tgt_input)   # 2. predict
loss = criterion(...)     # 3. how wrong?
loss.backward()           # 4. work out how to change every weight
optimizer.step()          # 5. change them a little
```

This repeats for 150 epochs over the 935 Complete rows. It saves the weights plus `"variant": "tinyllama"` and the settings, so the pipeline can rebuild the model.

4–6. It reads `dataset_v4_train.jsonl` and writes `trained_tinyllama.pt`. It uses `tokenizer`, `vocab_pool` and `decoder_tinyllama`.

7. Key concepts. Teacher forcing; epoch; batch; loss; cross-entropy; backpropagation; AdamW; `state_dict`; seed.

4.11 train_picker.py: teaches the word picker

1. Purpose. Trains the word picker to learn two things at once: the keep/drop/slot decision for each word, and the "add how?" flag for each sentence. It saves `trained_picker.pt`.

3. How it works. Each row must have exactly one label per word, or it's skipped. Then two losses are added together:

```
tag_logits, q_logits = model(input_ids)
tag_loss = tag_criterion(tag_logits.reshape(-1, tag_logits.size(-1)), labels.reshape(-1))
q_loss = q_criterion(q_logits, questions)
loss = tag_loss + q_loss
```

One `backward()` improves both heads. Settings match `train_distilbert.py` (100 epochs, seed 42).

4–6. It reads `labels_train.jsonl` (533 rows, 10 of them question rows) and writes `trained_picker.pt`. It uses `tokenizer`, `vocab_pool` and `word_picker`.

7. Key concepts. Multi-task learning. **Class imbalance:** only 1.9% of rows are "add how", yet the flag is right on all 294 held-out rows, including all 13 question rows.

4.12 slm_pipeline.py: the adviser's pipeline (new)

1. Purpose. The SLM organised exactly like the pipeline diagram: **one method per box**, and one method per path. Both demos and the conversation test run through it.

2. What it contains

Name	What it is
`LOOKUP_TABLE`, `SIGNS`, `MAX_WORDS`	The 88 known words, the 40 signs, the 16-word limit
`PEOPLE`, `QUESTIONS`, `STATES`, `ACTIVITIES`, `PLACES`	Groups built from `vocab_pool.CATEGORIES`
`CONTRACTIONS`, `PRONOUN_FITS`, `COVER_FORMS`	Tables for Normalization, Active Context and the safety net
`Turn`	What one pass produced: the box-by-box `steps`, plus the screen `output`
`ActiveContext`	The memory both paths share: `resolve_pronoun`, `note_hearing`, `fill_subject`, `note_deaf`, `reset`
`to_declarative()`, `covered()`	The answer-form and safety-net rules
`picker_training_words()`, `known_form()`	The words the picker was trained on, and the trained form of a synonym (used by Normalization)
`SLMPipeline`	Loads the two trained models; one method per box (see Part 2)
`non_deaf_to_deaf()`, `deaf_to_non_deaf()`	The two paths
`format_turn()`, `main()`	Printing, and an example conversation

3. How it works. `SLMPipeline()` loads `trained_picker.pt` and `trained_tinyllama.pt` (rebuilding each network from the settings saved inside the file), builds the tokenizer, and starts an empty `ActiveContext`. Each path calls the boxes in order, and records what each box produced in `turn.steps`, which is what the demos print. Part 2 covers every box's code.

The memory in detail (`ActiveContext`):

Stored	Set by	Used by
`person`	both paths: the last of mother/father/teacher/family/friend mentioned	he/she replacement; filling a missing subject

Stored	Set by	Used by
`last_was_question`, `last_question_words`	hearing turns	answer form; place answers to *where*
`addressed_to_you`	hearing turns: only the word `you`	stops a subject being added when the Deaf person was asked about themself

`reset()` clears all of these, which fixed the old bug where `last_was_question` survived a reset.

4. Inputs. Typed or spoken sentences; recognised signs. **5. Outputs.** A `Turn` with the screen text and the trace.

6. Connections. Uses `tokenizer`, `vocab_pool`, `onnx_config`, `make_labels`, `word_picker`, `decoder_tinyllama` and `speech_input`. Used by `full_demo`, `two_way_demo` and `Conversation_test`.

7. Key concepts

- **Classes and methods.** A class bundles data (the models, the memory) with the functions that use it.
- **State.** `ActiveContext` keeps information *between* turns.
- **Pipeline pattern.** Each stage takes the previous stage's output.

4.13 speech_input.py: Voice Recording and Speech to text (new)

1. Purpose. The microphone and the pre-trained speech-to-text model (Vosk) for the hearing → Deaf path.

2. What it contains:

- `SAMPLE_RATE = 16000`, `RECORD_SECONDS = 5.0` and `MODEL_DIR` (`models/vosk-model-small-en-us-0.15`)
- `SpeechUnavailable`: the error raised when something is missing
- `is_available()`, `record()` and `transcribe()`

3. How it works. See Part 2 (Voice Recording, Speech to text). The libraries are imported *inside* the functions, so everything else runs even when Vosk isn't installed. `is_available()` tells the demo whether to offer the microphone option.

Setup, once your group decides: `pip install vosk sounddevice`, then download `vosk-model-small-en-us-0.15.zip` (about 40 MB) from alphacephei.com/vosk/models and unzip it into `slm_training_v2/models/`.

Heads-up. Untested until installed. It records for a fixed 5 seconds; a "stop when silent" version is a possible improvement.

4.14 full_demo.py: the live demo with every box traced

1. Purpose. A live two-way conversation that prints every pipeline box as it runs. It's the file to use in front of your adviser.

2–3. How it works. It creates one `SLMPipeline`, then loops on a menu:

- `1` = the Deaf person signs
- `2` = type what the hearing person says
- `3` = speak into the microphone (only shown once Vosk is installed)
- `r` = reset the memory
- `q` = quit

Each turn calls `deaf_to_non_deaf()` or `non_deaf_to_deaf()`. It prints `format_turn(turn)` (the numbered boxes), then the reader's screen and the memory. It contains no pipeline logic of its own.

Real output of option 2 with `Where is your mother?` (the first 8 of 11 boxes):

```
hearing -> Deaf
   1. Voice Recording    skipped: typed input
   2. Speech to text     typed: 'Where is your mother?'
   3. Normalization      where is your mother
   4. Reduction          where is your mother   [nothing removed]
   5. Tokenize           [85, 41, 91, 51]
   6. Embedding          4 words x 64 numbers
   7. Contextualize      2 transformer blocks: each word now reflects the whole sentence
   8. Space Restriction  where=slot 1, is=drop (no sign), your=drop, mother=slot 2; ...
```

Heads-up. The earlier bugs of this file (the "your" memory bug, the reset bug, the display glitch) are gone, because its logic now lives in `slm_pipeline.py`.

4.15 two_way_demo.py: the compact demo

1. Purpose. The same pipeline and memory as `full_demo.py`, showing only the two screens and the memory. Options: `1` sign, `2` speak (typed), `r` reset, `q` quit.

Real output: `How is your father?` → Deaf screen `how father`; then signs `fine` → hearing screen **`father is fine`**. Before the fix, this gave `i am fine`.

4.16 Conversation_test.py: scripted conversations against the dataset

1. Purpose. Runs 7 realistic conversations (51 turns) through the pipeline, and compares each answer with the dataset's answer for that input.

3. How it works. For each turn it clears the memory (so each answer is judged on its own), runs `deaf_to_non_deaf()` or `non_deaf_to_deaf()`, and labels the result MATCH, DIFFER, or NEW (not in the dataset; you judge it).

Real results: MATCH 45/51, DIFFER 0/51, NEW 6/51. The Deaf side uses 30 of the 40 signs. The 6 NEW cases:

Input	Output	Sensible?
hear: `is your family okay`	`family`	✗ should be `how family`
sign: `my study school`	`i am studying at school`	✓
sign: `my learn home`	`i am learning at home`	✓
sign: `my eat home`	`i am eating at home`	✓
sign: `my drive school`	`i am driving at school`	✓
hear: `what is your family doing`	`what family`	✓ reasonable

Heads-up. Because the memory is cleared each turn, this doesn't test Active Context. Most MATCH rows were in training, so the 6 NEW rows are the real test.

4.17 compare.py: Reduction accuracy on held-out sentences

1. Purpose. Measures the word picker on `labels_test.jsonl`: 294 phrasings it never trained on.

3. How it works. For each row it runs `picker_predict()` and compares the result with the answer (ignoring punctuation). It also checks the question flag, and whether any word was "invented".

Real results: picker exact match **290 / 294 = 98.6%**, question flag 294 / 294. The 4 misses:

Said	Expected	Picker gave
`do you want help`	`help`	`want help` (arguably fine)
`are you fine`	`how you`	`how fine you`
`it is nice to meet you`	`nice`	`nice you`
`i do not understand`	`understand`	`no understand` (the dataset label is the wrong one)

Heads-up. Its "decoder" column now shows 0/294, because TinyLlama is trained for Sentence Completion only. That's not a fair comparison, so don't report it. The earlier fair comparison was 97.8% (picker) vs. 88.0% (decoder). Its `load_checkpoint` and `decoder_predict` helpers are no longer used by the demos.

Part 5: Full walkthrough of one conversation

This is the real output of `python slm_pipeline.py`. `full_demo.py` prints the same trace, one turn at a time.

Turn 1: the hearing person says "Where is your mother?"

```
hearing -> Deaf
   1. Voice Recording    skipped: typed input
   2. Speech to text     typed: 'Where is your mother?'
   3. Normalization      where is your mother
   4. Reduction          where is your mother   [nothing removed]
   5. Tokenize           [85, 41, 91, 51]
   6. Embedding          4 words x 64 numbers
   7. Contextualize      2 transformer blocks: each word now reflects the whole sentence
   8. Space Restriction  where=slot 1, is=drop (no sign), your=drop, mother=slot 2;
                         add 'how': no -> where mother
   9. Active Context     remembers mother
  10. Transcribed Words  where mother
  11. Display            Deaf person's screen
  => screen: 'where mother'
```

What happened at each box:

- **Normalization** removed the `?`. This is the bug that used to make this sentence give only `where`.
- **Reduction** found every word in the lookup table.
- **Embedding** turned the 4 IDs into 4 × 64 numbers.
- **Contextualize** let every word read the whole sentence.
- **Space Restriction** kept `where` (slot 1) and `mother` (slot 2), from the scores shown in Part 2.
- **Active Context** stored: person = mother, question word = where, and not asked to the Deaf person, because the word was `your`, not `you`.

Turn 2: the Deaf person signs "home"

```
Deaf -> hearing
   1. Transcribed Words    home
   2. Tokenize             [37]
   3. Embedding            1 sign x 64 numbers
   4. Active Context       added 'mother' from earlier
   5. Sentence Completion  mother is home   [answer form: 'is mother home' -> 'mother is home']
   6. Display              hearing person's screen
  => screen: 'mother is home'
```

What happened at each box:

- **Active Context** saw a bare place answering a *where* question about mother, so it joined mother's vector to the front of the sign vectors.
- **Sentence Completion** wrote `is` → `mother` → `home` → `<eos>`, each at 100% confidence.
- Because this was a reply to a question, the answer form turned it into **`mother is home`**. (The old code answered `i am home`.)

The rest of the conversation

Turn	Input	Key boxes	Screen
3 hearing	`Is she okay?`	Space Restriction adds `how`; Active Context: she → mother	`how mother`
4 Deaf	`fine`	Active Context adds mother (a state needs a subject)	`mother is fine`
5 hearing	`How is your father?`	Active Context now remembers father	`how father`
6 Deaf	`sick`	Active Context adds father	`father is sick`
7 hearing	`I don't know where he is.`	Normalization: don't → do not; Space Restriction: not → `no`; Active Context: he → father	`no where father`
8 Deaf	`drive mother where`	Active Context: a person was signed, nothing added	`where is mother driving`

Reduction removing words, from another run: `Where did your mommy park the car?`

- Normalization: mommy → mother
- Reduction removed `did, park, car`
- Result: `where mother`

Part 6: Defense preparation

A 1-minute summary you can say

> "Our SLM follows the pipeline in this diagram, and the code has one function per box, in the same order.
>
> When the hearing person speaks, the speech is transcribed, normalized (punctuation removed, short forms like 'don't' written out, synonyms like 'mommy' mapped to 'mother'), and reduced to the 88 words in our lookup table. The words are tokenized into IDs and embedded as 64 numbers each. Two transformer blocks contextualize them, so every word reflects the whole sentence. Space Restriction then narrows the sentence down to our 40 signs: our trained word picker scores each word as drop or keep-in-slot, only signs can survive, and the Hungarian algorithm gives each kept word its own slot. Active Context remembers who we're talking about and replaces 'he' or 'she' with that person. The result goes to the Deaf person's screen. On 294 unseen sentences it gets 290 right.
>
> When the Deaf person signs, the recognised signs are tokenized and embedded. Active Context adds the remembered person if the signs need a subject, and our trained TinyLlama model completes the sentence word by word. A safety check makes sure no sign was lost."

10 questions your adviser might ask

1. Walk me through your pipeline in the code.

`slm_pipeline.py` has one method per box, and two methods that call them in the diagram's order: `non_deaf_to_deaf()` and `deaf_to_non_deaf()`. `full_demo.py` prints each box's output as it runs, so you can check it against the diagram live.

2. Why is there no Restructure step?

It's not in the pipeline, per the adviser. We verified this changes nothing: TinyLlama embeds the signs without position, so it writes the identical sentence whatever order the signs arrive in. It did so on all 979 sign inputs.

3. Why does Reduction keep 88 words instead of only the 40 signs?

The word picker needs some non-sign words to understand the sentence; `is` at the start is how it spots the question in `is father okay`. Reduction removes every word outside our lookup table. Narrowing down to the 40 signs happens in Space Restriction, after the model has read the context.

4. What exactly happens in Embedding?

Each word ID selects a row of 64 learned numbers from a word table. For speech, a row from a position table is added and the result is normalised. Real example: `mother` → `[+1.40, -1.71, +0.65, +1.48, ...]`. The code is `DistilBertEmbeddings.forward()` in `encoder_distilbert.py`.

5. What does Contextualize add?

Self-attention lets every word look at every other word, so each word's numbers come to reflect the whole sentence. The same word `mother` ends up with different numbers in `where is your mother` and in `mother is sick`.

6. What is Space Restriction, and where is the Hungarian algorithm?

Space Restriction turns the model's scores into signs. The tag head scores each word (drop, or keep in slot 1–16). Rules keep only signs and turn negations into `no`. `linear_sum_assignment` (the Hungarian algorithm) then gives each kept word its own slot, choosing the one-to-one match with the highest total score.

7. What does Active Context do in each direction?

- Hearing → Deaf: it remembers the person and replaces he/she with them.
- Deaf → hearing: it adds the remembered person when the signs need a subject (`where`, `fine`, or `home` answering *where*), and not when the person was asked about themself.

These are hand-written rules, not a trained model.

8. Why a word picker for Reduction instead of a generator?

A generator can invent words nobody said. The picker only keeps or drops spoken words (plus "how"), so it can't invent anything. It scores 98.6% on unseen sentences, with 0 invented words.

9. How do you know it isn't memorising?

The generator holds out whole phrasings. The picker gets 290 of 294 unseen phrasings right, and the conversation test gives sensible answers for combinations never in the data. Be honest that the held-out set for Sentence Completion is small (46 rows).

10. How does speech get into the system?

Through the Voice Recording and Speech to text boxes, using Vosk: a free, pre-trained, offline speech recogniser that runs on a Raspberry Pi. The code is ready in `speech_input.py`; installation is pending the group's decision, so the demo currently uses typed text.

Weak spots: what was fixed and what is still open

Fixed in this update

#	Problem	How it was fixed
1	Punctuation broke Reduction	Normalization removes punctuation first
2	Apostrophes shifted every word's score	Normalization writes out short forms; words and scores always line up
3	17+ spoken words crashed	Input cut at 16 words
4	"your" treated as "you" by the memory	Only the word `you` counts as asking the Deaf person
5	Place answers got no subject	A bare place answering *where* now takes the remembered person
6	Memory reset left `last_was_question` behind	`ActiveContext.reset()` clears everything
7	Demos duplicated the pipeline logic	Demos are now thin screens over `slm_pipeline.py`
8	`mom`, `dad`, `house` were never in the picker's training sentences, so `Is your mom at home?` gave just `home`	Normalization maps synonyms the picker never saw to the word it knows. Now `mother home`

Still open

#	Limitation	What to say or do
9	TinyLlama can't see sign order	"mother help father" = "father help mother". Fix: a position table for the signs, then retrain

#	Limitation	What to say or do
10	Small held-out set for Sentence Completion (46 rows)	Present the Reduction held-out result (294 rows) as the main evidence
11	Speech to text not installed yet	Pending the group's decision; typed input for now
12	Some lookup-table words never appear in the picker's training sentences: `what`, `finish`, `going`, `and`, `he`, `she`, `know`, `really`, `this`, `today`	Its decisions on those words are guesses. Add training sentences that use them, then retrain
13	The wording "is your ... okay" was never trained	`Is your mother okay?` → `mother` (missing `how`). Add "is your {person} okay" to the generator, then retrain
14	The safety net checks presence, not meaning	It uses a 4-letter prefix match (`no` would match `now`)
15	One dataset label drops a negation	`i do not understand` → `understand`; fix the label in the generator
16	Several boxes are rules, not learning	Active Context, the answer form, the safety net and the sign filter are rules; say so
17	He/she uses fixed lists	Only he/she are handled (not "they" or "him"); family is never matched
18	Leftovers	"39 signs" comments; `make_labels` docstring says v3; unused `PRONOUN_OF`; ONNX/Pi files out of date

What to do next

1. Decide on speech to text with your group. If yes, install Vosk and test with a real microphone.
2. Improve the training sentences in `generate_dataset_v4.py`: fix the negation label, and add sentences that use the untrained words (#12) and the "is your ... okay" wording (#13). Then regenerate, run `make_labels.py`, and retrain the picker.
3. Tidy the leftover comments.
4. When the Pi arrives, update the ONNX export in `pi_deployment_later/` to the new pipeline.